Temporal-Difference (TD) Learning and Function Approximation Methods

What is Reinforcement Learning (RL)?

In Reinforcement Learning, an agent (like a robot or game-playing AI) learns by interacting with an environment to maximize rewards — just like how a child learns not to touch a hot stove after getting burned.

The agent needs to learn: *"How good is it to be in a particular situation (state)?"*

The 3 Methods: DP, MC, and TD

Method	Full Name	How It Learns	Needs environment model?	Waits for episode to end?
DP	Dynamic Programming	Uses a complete map of the world	✅ Yes	✅ Yes
MC	Monte Carlo	Plays full games, learns from final result	❌ No	✅ Yes
TD	Temporal Difference	Learns *step by step*, mid-game	❌ No	❌ No

Intuitive Analogy: The Driving Student 🚗

Imagine you're learning to drive and want to predict if you'll reach your destination safely.

- **MC approach:** You wait until the entire trip is over, then reflect on every decision you made. Very accurate, but you can only learn after the journey ends.
- **DP approach**: You have a perfect GPS map. You can calculate the best route before even starting — but this requires a complete model of all roads and traffic.
- **TD approach**: After each traffic light, you update your estimate: ***"Hmm, that lane is moving well, I think I'll arrive on time."*** You don't wait for the trip to end; you adjust your guess at every step.

That **step-by-step correction is the essence of Temporal Difference** — the "difference" between your prediction now and your prediction at the next moment in time

The Core Idea: Bootstrapping

The key concept TD borrows from DP is called bootstrapping — updating your guess based on another guess, rather than waiting for the true final answer.

🎯 Think of it like this: *You guess today's weather will be sunny. Tomorrow morning, you update that guess based on what the clouds look like right now — not by waiting for the whole day to pass.*

This is the TD update formula in plain English:

$$\text{New Estimate} = \text{Old Estimate} + \alpha \times \underbrace{(\text{Reward} + \text{Next State Value} - \text{Old Estimate})}_{\text{TD Error (the "Temporal Difference")}}$$

The TD Error is literally the difference between what you expected and what the next step suggests — hence "Temporal Difference."

Why TD is Powerful

TD combines the best of both worlds:

- **From MC:** No need for a model of the environment — it learns from real experience.
- **From DP:** Doesn't wait for the final outcome — it bootstraps (updates guess with guess).

Can work on ongoing, never-ending tasks (like controlling a robot continuously), unlike MC which needs an episode to finish.

The Core Difference: Patience

The single biggest difference between MC and TD is when they learn.

Think of two students studying for an exam:

- 🔴 MC student reads the entire chapter first, then tries to understand everything at the end.
- 🔵 TD student pauses after every paragraph, reflects, corrects their understanding, and moves on.

	Monte Carlo method	TD method
Sequence completeness	Must wait until end of episode Can only learn from complete sequences Only works for episodic environments	Can learn online after every step Can learn from incomplete sequences Works in continuing environments
Variance and bias	High variance, zero bias (even with function approximation) Not very sensitive to initial value	Low variance, some bias Usually more efficient than MC TD(0) converges to True $V\pi(s)$, but does not always converge with function approximation More sensitive to initial value
Markov property	Does not exploit Markov property MC converges to solution with minimum mean-squared error	Usually more efficient in Markov environments TD(0) converges to solution of max likelihood Markov model

Properties	DP	MC	TD(0)
Sampling		√	√
Bootstrapping	√		√
Usable when no models of current domain		√	√
Handles continuing (non-episodic) domains	√		√
Handles non-Markovian domains		√	
Converges to true value in limit (Markov)	√	√	√
Unbiased estimate of value	Exact	√ (first-visit)	

Three Ways to Estimate V(s)

Way 1 — Dynamic Programming (DP)

When you have a complete model of the environment (you know all transitions and rewards), you can use the Bellman equation directly:

$$V^{\pi}(S_t) \leftarrow \mathbb{E}_{\pi}[R_{t+1} + \gamma V^{\pi}(S_{t+1})]$$

Way 2 — Monte Carlo (MC)

When you have no model, you run full episodes and average the real returns:

$$V^{\pi}(S_t) \leftarrow V^{\pi}(S_t) + \alpha(G_t - V^{\pi}(S_t))$$

Way 3 — TD Learning (the star of the show ⭐)

TD uses a shortcut: instead of waiting for the real G_t, it estimates it using just the immediate reward + the value of the next state:

$$V^{\pi}(S_t) \leftarrow V^{\pi}(S_t) + \alpha\big(R_{t+1} + \gamma V^{\pi}(S_{t+1}) - V^{\pi}(S_t)\big)$$

The Three Formulas Side by Side

$$\underbrace{V^\pi(S_t) \leftarrow \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1})]}$$

DP: uses full model, no sampling

$$\underbrace{V^\pi(S_t) \leftarrow V^\pi(S_t) + \alpha(G_t - V^\pi(S_t))}$$

MC: waits for real return G_t

$$\underbrace{V^\pi(S_t) \leftarrow V^\pi(S_t) + \alpha\big(R_{t+1} + \gamma V^\pi(S_{t+1}) - V^\pi(S_t)\big)}$$

TD: uses one-step estimate, updates immediately

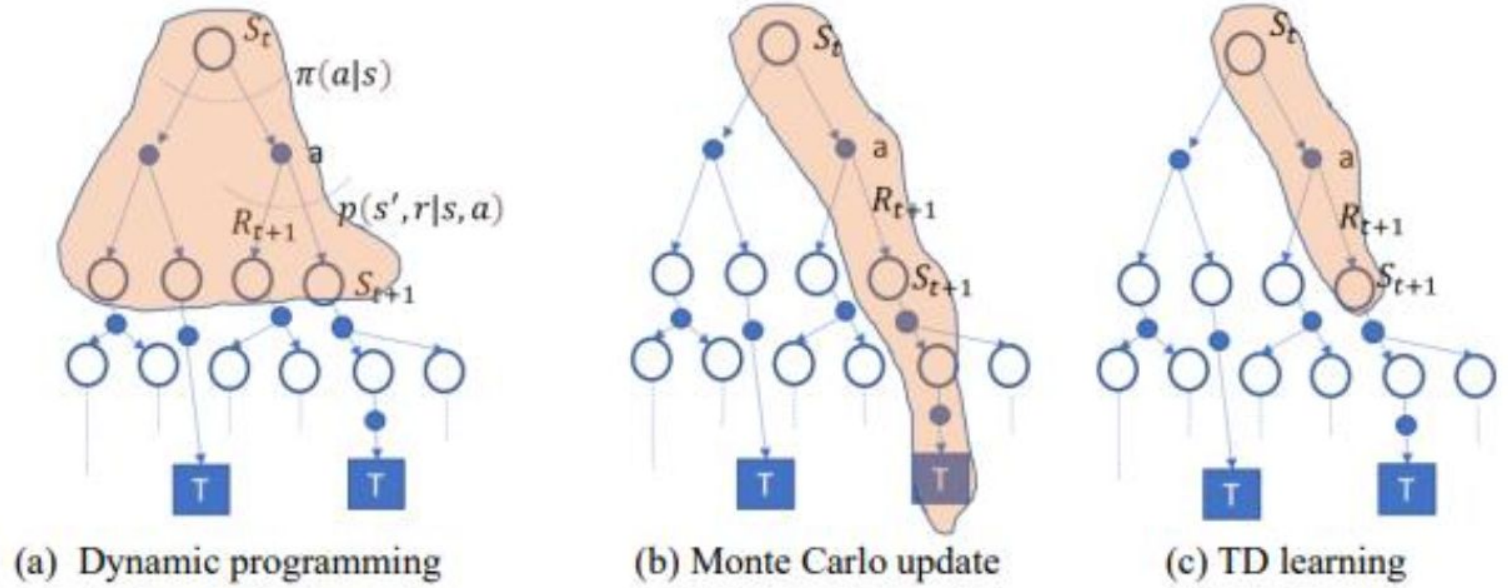
Why TD Matters Beyond Just V(s)

TD learning is foundational because the same idea powers the most famous RL algorithms you'll learn next:

- **SARSA** — TD applied to action-values, on-policy
- **Q-Learning** — TD applied to action-values, off-policy (the algorithm behind many game-playing AIs)

Both are directly inspired by the TD update rule above.

Comparison of TD, DP and MC



TD Learning

Imagine you're trying to estimate how long your commute will take each day.

- Each morning, you make a guess (your current estimate)
- You start driving, take one step, and check traffic conditions
- You revise: "Based on traffic right now + my expected remaining time" → that's your new target
- The difference between your new target and your old guess = your error
- You nudge your original guess a little toward the new target

That's exactly what TD learning does — in formula form.

$$V^{\pi}(S_t) \leftarrow V^{\pi}(S_t) + \alpha\big(\underbrace{R_{t+1} + \gamma V^{\pi}(S_{t+1})}_{\text{TD Target}} - V^{\pi}(S_t)\big)$$

$$V^\pi(S_t) \leftarrow V^\pi(S_t) + \alpha\Big(\underbrace{R_{t+1} + \gamma V^\pi(S_{t+1})}_{\text{TD Target}} - V^\pi(S_t)\Big)$$

Term	What It Means
$V^\pi(S_t)$	Your **current estimate** of state S_t
R_{t+1}	The **actual reward** received after one step
$V^\pi(S_{t+1})$	Your **estimate** of the *next* state's value
γ	**Discount factor** — future rewards worth a bit less
$R_{t+1} + \gamma V^\pi(S_{t+1})$	**TD Target** — your *updated* guess
$\text{TD Target} - V^\pi(S_t)$	**TD Error** δ_t — how wrong you were
α	**Learning rate** — how aggressively you correct yourself

TD Target 🎯

$$\text{TD Target} = R_{t+1} + \gamma V^{\pi}(S_{t+1})$$

This is your **revised estimate** of how good state S_t is, constructed from:

- **Real data**: the actual reward R_{t+1} you just received
- **Bootstrapped estimate**: your current guess about S_{t+1}

TD Error ⚡

$$\delta_t = R_{t+1} + \gamma V^\pi(S_{t+1}) - V^\pi(S_t)$$

This is the **surprise signal** — how much reality differed from your expectation.

- $\delta_t > 0$: Things went *better* than expected → increase $V(S_t)$ ⬆️
- $\delta_t < 0$: Things went *worse* than expected → decrease $V(S_t)$ ⬇️
- $\delta_t = 0$: Your prediction was perfect → no update needed ✅

The Update in Plain English

Keep your old estimate, but nudge it a little bit (α) in the direction of your correction (δ_t).

$$V^\pi(S_t) \leftarrow V^\pi(S_t) + \alpha\big(\underbrace{R_{t+1} + \gamma V^\pi(S_{t+1})}_{\text{TD Target}} - V^\pi(S_t)\big)$$

$$\underbrace{V^\pi(S_t)}_{\text{new estimate}} \leftarrow \underbrace{V^\pi(S_t)}_{\text{old estimate}} + \underbrace{\alpha}_{\text{step size}} \times \underbrace{\delta_t}_{\text{TD error}}$$

Why is it called TD(0)?

The "0" in TD(0) means you **look 0 steps beyond the next state** — you only use a one-step return.

You could also do:

- TD(1) = look 2 steps ahead
- TD(n) = look n steps ahead
- TD(∞) = look all the way to terminal state = same as Monte Carlo!

So MC is actually a special case of TD where n=∞. TD(0) is simply the most basic, one-step version.

TD(0) Learning Algorithm

TD(0) learning algorithm

Input : the policy π to be evaluated, learning rate α

Output: value function V^{π}

Initialize $V^{\pi}(s) = 0 \; for \; all \; s$

for each episode **do**

for each step t of episode, state S_t is not terminal **do**

- Take action A_t by following policy π
- Observe sample tuple $(S_t, A_t, R_{t+1}, S_{t+1})$
- $V^{\pi}(S_t) \leftarrow V^{\pi}(S_t) + \alpha\left(\boldsymbol{R_{t+1}} + \boldsymbol{\gamma V^{\pi}(S_{t+1})} - V^{\pi}(S_t)\right)$
- $S_t \leftarrow S_{t+1}$

end

end

Why is it Called TD(0)?

The "0" means zero additional steps of lookahead beyond the next state S_{t+1}.

It only uses the immediate next state to bootstrap — making it the simplest and most computationally efficient form of TD learning.

As you'll learn soon, SARSA and Q-Learning are direct extensions of this exact algorithm

Task: Apply the TD(0) algorithm for 3 complete episodes. After each episode, state the updated value function. Show every TD Target, TD Error, and update calculation.

An agent navigates a **linear 4-state MDP** with states S_1, S_2, S_3, S_T where S_T is the **terminal state**. The agent always follows a fixed policy π: move right.

Transition and Reward structure:

Transition	Reward
$S_1 \to S_2$	0
$S_2 \to S_3$	0
$S_3 \to S_T$	+10

Given parameters:

- Learning rate $\alpha = 0.1$
- Discount factor $\gamma = 0.9$
- Initial values: $V(S_1) = V(S_2) = V(S_3) = 0, V(S_T) = 0$ (always)

🔁 Episode 1

Starting values: $V(S_1) = 0,\ V(S_2) = 0,\ V(S_3) = 0$

Step t=0: State S_1, move to S_2, Reward = 0

$$\text{TD Target} = R_{t+1} + \gamma V(S_2) = 0 + 0.9 \times 0 = 0$$

$$\delta_0 = 0 - V(S_1) = 0 - 0 = 0$$

$$V(S_1) \leftarrow 0 + 0.1 \times 0 = \boxed{0}$$

Step t=1: State S_2, move to S_3, Reward = 0

$$\text{TD Target} = 0 + 0.9 \times V(S_3) = 0 + 0.9 \times 0 = 0$$

$$\delta_1 = 0 - V(S_2) = 0 - 0 = 0$$

$$V(S_2) \leftarrow 0 + 0.1 \times 0 = \boxed{0}$$

Step t=2: State S_3, move to S_T, Reward = +10

$$\text{TD Target} = 10 + 0.9 \times V(S_T) = 10 + 0 = 10$$

$$\delta_2 = 10 - V(S_3) = 10 - 0 = 10$$

$$V(S_3) \leftarrow 0 + 0.1 \times 10 = \boxed{1.0}$$

After Episode 1: $V(S_1) = 0,\ V(S_2) = 0,\ V(S_3) = 1.0$

🔁 Episode 2

Starting values: $V(S_1) = 0,\ V(S_2) = 0,\ V(S_3) = 1.0$

Step t=0: State S_1, move to S_2, Reward = 0

$$\text{TD Target} = 0 + 0.9 \times V(S_2) = 0 + 0.9 \times 0 = 0$$

$$\delta_0 = 0 - 0 = 0$$

$$V(S_1) \leftarrow 0 + 0.1 \times 0 = \boxed{0}$$

Step t=1: State S_2, move to S_3, Reward = 0

$$\text{TD Target} = 0 + 0.9 \times V(S_3) = 0 + 0.9 \times 1.0 = 0.9$$

$$\delta_1 = 0.9 - V(S_2) = 0.9 - 0 = 0.9$$

$$V(S_2) \leftarrow 0 + 0.1 \times 0.9 = \boxed{0.09}$$

Step t=2: State S_3, move to S_T, Reward = +10

$$\text{TD Target} = 10 + 0.9 \times 0 = 10$$

$$\delta_2 = 10 - V(S_3) = 10 - 1.0 = 9.0$$

$$V(S_3) \leftarrow 1.0 + 0.1 \times 9.0 = \boxed{1.9}$$

After Episode 2: $V(S_1) = 0,\ V(S_2) = 0.09,\ V(S_3) = 1.9$

🔁 Episode 3

Starting values: $V(S_1) = 0,\ V(S_2) = 0.09,\ V(S_3) = 1.9$

Step t=0: State S_1, move to S_2, Reward = 0

$$\text{TD Target} = 0 + 0.9 \times V(S_2) = 0 + 0.9 \times 0.09 = 0.081$$

$$\delta_0 = 0.081 - V(S_1) = 0.081 - 0 = 0.081$$

$$V(S_1) \leftarrow 0 + 0.1 \times 0.081 = \boxed{0.0081}$$

Step t=1: State S_2, move to S_3, Reward = 0

$$\text{TD Target} = 0 + 0.9 \times V(S_3) = 0 + 0.9 \times 1.9 = 1.71$$

$$\delta_1 = 1.71 - V(S_2) = 1.71 - 0.09 = 1.62$$

$$V(S_2) \leftarrow 0.09 + 0.1 \times 1.62 = 0.09 + 0.162 = \boxed{0.252}$$

Step t=2: State S_3, move to S_T, Reward = +10

$$\text{TD Target} = 10 + 0.9 \times 0 = 10$$

$$\delta_2 = 10 - V(S_3) = 10 - 1.9 = 8.1$$

$$V(S_3) \leftarrow 1.9 + 0.1 \times 8.1 = 1.9 + 0.81 = \boxed{2.71}$$

After Episode 3: $V(S_1) = 0.0081,\ V(S_2) = 0.252,\ V(S_3) = 2.71$

📊 Summary of Value Updates

State	After Episode 1	After Episode 2	After Episode 3
$V(S_1)$	0	0	0.0081
$V(S_2)$	0	0.09	0.252
$V(S_3)$	1.0	1.9	2.71

The Problem with TD(0)

TD(0) only looks one step ahead before updating. This is fast, but the reward signal propagates very slowly — as we saw in the numerical example, S_1 took 3 full episodes to even start updating.

The question is: Can we look more than one step ahead, but still not wait for the full episode like MC?

Enter n-Step TD

Instead of just one step, we can look n steps ahead before updating:

Method	How far it looks	Name
TD(0)	1 step	One-step TD
TD(n=2)	2 steps	Two-step TD
TD(n=∞)	Full episode	= Monte Carlo!

🔭 Analogy: TD(0) is like checking traffic at the next block only. n-step TD is like checking the next n blocks. MC is checking all the way to your destination.

But now a new problem arises: which value of n should you pick?

TD(λ) — The Elegant Solution

Instead of picking one n, TD(λ) uses a weighted average of ALL n-step returns at once, controlled by the parameter **λ∈[0,1]**

The weight given to the n-step return decreases geometrically:

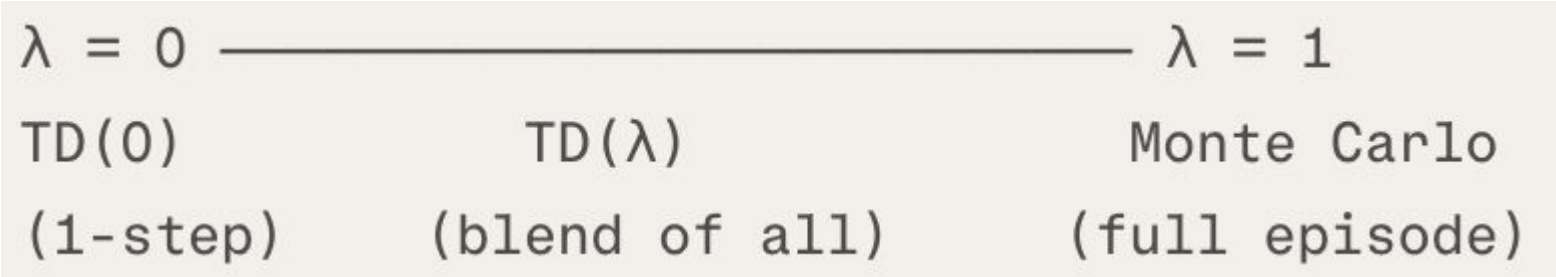
$$G_t^\lambda = (1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}G_t^{(n)}$$

- Short-term returns (small n) get **more weight**
- Long-term returns (large n) get **less weight**, decaying by λ each step

🎯 Analogy: You're estimating how long a journey will take. You trust the road conditions 1 km ahead the most, 2 km ahead a little less, 3 km ahead even less... TD(λ) combines all of these estimates into one smart prediction.

The λ Dial 🎛️

λ is literally a dial between TD(0) and MC:



- **λ=0:** Only the 1-step return matters → pure TD(0)
- **λ=1**: All future steps matter equally → equivalent to Monte Carlo
- **0<λ<1**: A smart blend — gets the best of both worlds

In practice, values like λ=0.9 or λ=0.95 often work well.

Two Ways to Implement TD(λ)

🔭 Forward View (Theoretical)

- You stand at state S_t and look forward into the future, computing all n-step returns, then blend them.
- Clean mathematically
- **Problem**: You still need to wait until the episode ends to get those future returns ❌
- **Not directly implementable online**

🔄 Backward View — Eligibility Traces (Practical)

- Instead of looking forward, you look **backward** using a concept called eligibility traces.
- TD error "shouts backwards" through time. States visited recently hear the shout loudly. States visited long ago barely hear it.

Eligibility Traces in Simple words

Imagine a detective investigating who caused a crime (the reward/punishment).

- States visited recently are most eligible for credit/blame
- States visited frequently are also more eligible
- States visited long ago have low eligibility — their trace has faded

This is exactly how eligibility traces work — they track recency and frequency of state visits to distribute credit properly.

Part 1: Building the n-Step Return

Recall that TD(0) uses only 1 reward + estimate of next state. What if we collected more rewards before bootstrapping? That's the n-step return.

The n-Step Return Formula Explained

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

Think of it as:

"Collect n real rewards, then plug in an estimate for everything after that."

Part 2: The Problem — Which n to Choose?

No single value of n is always best.

Small n (like 1): Fast but slow credit propagation, high bias

Large n (like 100): Accurate but slow, high variance, must wait longer

TD(λ) solves this elegantly: instead of picking one n, it uses **a weighted average of ALL n-step returns simultaneously**

Part 3: The λ-Return — The Core Formula

$$G_t^\lambda = (1-\lambda)\sum_{i=1}^{n} \lambda^{i-1} G_t^{(i)} + \lambda^n G_t^{(\infty)}$$

🎂 Analogy: Imagine cutting a cake in half repeatedly. The first slice (1-step return) is the biggest. The second slice (2-step return) is half of what remained. And so on. The total always adds up to exactly one full cake.

λ is literally a smoothly adjustable dial between TD and MC.

Part 4: The Offline λ-Return Algorithm

The update rule for TD(λ) using the λ-return is:

$$V(S_t) \leftarrow V(S_t) + \alpha\big(G_t^\lambda - V(S_t)\big)$$

📚 Analogy: It's like a student who takes notes during the entire lecture (collects all rewards), then goes home and studies everything at once (applies all updates at episode end) — rather than studying after each slide.

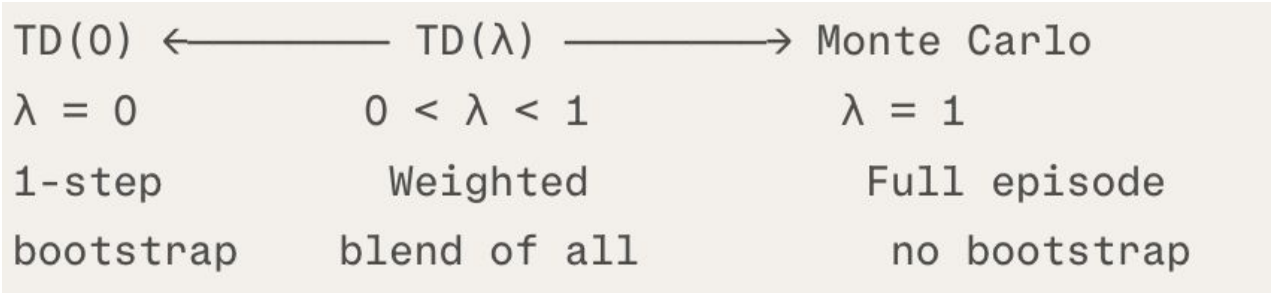
Part 5: Why Forward View is a Problem

"The forward view of TD(λ) is not directly implementable online because it is non-causal"

Non-causal means: at time t, you need to know what happens at t+1, t+2, t+3... which are in the **future**. You can't know them yet!

This is why the **Backward View (Eligibility Traces)** was invented — it achieves the exact same mathematical result as the forward view, but works online, step by step, without needing to see the future.

The Complete TD(λ) Picture



Forward view: Mathematically clean, but offline only

Backward view (eligibility traces): Practically implementable, online.

The Core Problem: Who Deserves Credit?

Imagine a robot navigates a maze and finally receives a big reward at the end. The question is: **which states along the way actually deserve credit for that reward?**

This is called the **credit assignment problem**. Eligibility Traces solve it beautifully.

What is an Eligibility Trace?

Every single state (s) carries an extra memory variable called its eligibility trace $e_t(s)$.

Think of it like a glow attached to each state:

- When you visit a state → its glow brightens up ✨
- When you don't visit it → its glow fades away slowly 🌑

When a reward signal arrives → states that are currently glowing get updated proportionally

The Eligibility Trace Formula

$$e_t(s) = \begin{cases} \gamma\lambda \cdot e_{t-1}(s) & \text{if } s \neq S_t \\ \gamma\lambda \cdot e_{t-1}(s) + 1 & \text{if } s = S_t \end{cases}$$

Let's decode each case:

Case 1: State NOT visited this step ($s \neq S_t$)

$$e_t(s) = \gamma\lambda \cdot e_{t-1}(s)$$

The eligibility just **decays** by the factor $\gamma\lambda$. Since both $\gamma < 1$ and $\lambda < 1$, this decay is fast.

Case 2: State IS visited this step ($s = S_t$)

$$e_t(s) = \gamma\lambda \cdot e_{t-1}(s) + 1$$

The eligibility decays (as usual) but then **+1 is added** — the trace gets a fresh boost!

Why γλ Specifically?

Factor	Meaning
γ (discount)	Future rewards are worth less → distant states matter less
λ (trace decay)	Controls how far back credit can flow → small λ = short memory
$\gamma\lambda$ combined	Both time and relevance decay together

λ=0: Trace dies instantly after one step → only the most recent state gets credit → TD(0)

λ=1: Trace never decays (fully persistent) → all visited states get credit → Monte Carlo

How the Update Works

At every single step, ALL states get updated simultaneously using their eligibility:

$$V(S_t) \leftarrow V(S_t) + \alpha \cdot \underbrace{\delta_t}_{\text{TD error}} \cdot \underbrace{e_t(s)}_{\text{eligibility}}$$

🔊 *"At each moment, we compute a TD error and shout it backwards to every state we've visited — but states visited recently/frequently hear it louder than states visited long ago."*

Eligibility = Recency × Frequency

Dimension	Effect on trace
Recency	States visited recently have higher trace (haven't decayed much yet)
Frequency	States visited often have accumulated more +1 boosts

TD(λ) Backward Algorithm

Initialize $V(s)$ arbitrarily and $e(s) = 0$, for all $s \in \mathcal{S}$
Repeat (for each episode):
  Initialize s
  Repeat (for each step of episode):
    $a \leftarrow$ action given by π for s
    Take action a, observe reward, r, and next state, s'
    $\delta \leftarrow r + \gamma V(s') - V(s)$
    $e(s) \leftarrow e(s) + 1$
    For all s:
      $V(s) \leftarrow V(s) + \alpha \delta e(s)$
      $e(s) \leftarrow \gamma \lambda e(s)$
    $s \leftarrow s'$
  until s is terminal

Task: Apply the TD(λ) Online Backward View algorithm for one complete episode. Show every eligibility trace update, TD error, and value update at each step.

An agent navigates a **linear 4-state MDP**: $S_1 \to S_2 \to S_3 \to S_T$ (terminal).

Rewards:

Transition	Reward
$S_1 \to S_2$	0
$S_2 \to S_3$	0
$S_3 \to S_T$	+10

Given parameters:

- Learning rate $\alpha = 0.1$
- Discount factor $\gamma = 0.9$
- Trace-decay parameter $\lambda = 0.5$
- Initial values: $V(S_1) = V(S_2) = V(S_3) = 0$
- Initial eligibility traces: $e(S_1) = e(S_2) = e(S_3) = 0$

Episode Start
Initial state of everything:

	$V(S_1)$	$V(S_2)$	$V(S_3)$	$e(S_1)$	$e(S_2)$	$e(S_3)$
Start	0	0	0	0	0	0

⏱ Step 1: At S_1, move to S_2, reward = 0
① Compute TD Error δ:

$$\delta = r + \gamma V(S_2) - V(S_1) = 0 + 0.9 \times 0 - 0 = 0$$

② Boost eligibility of current state S_1:

$$e(S_1) \leftarrow 0 + 1 = 1$$

Traces before update: $e(S_1) = 1,\ e(S_2) = 0,\ e(S_3) = 0$
③ Update ALL values using $V(s) \leftarrow V(s) + \alpha\delta e(s)$:

$$V(S_1) \leftarrow 0 + 0.1 \times 0 \times 1 = 0$$

$$V(S_2) \leftarrow 0 + 0.1 \times 0 \times 0 = 0$$

$$V(S_3) \leftarrow 0 + 0.1 \times 0 \times 0 = 0$$

> 💡 $\delta = 0$, so no state changes — but the trace is now set!

④ Decay ALL traces by $\gamma\lambda = 0.9 \times 0.5 = 0.45$:

$$e(S_1) \leftarrow 0.45 \times 1 = 0.45$$

$$e(S_2) \leftarrow 0.45 \times 0 = 0$$

$$e(S_3) \leftarrow 0.45 \times 0 = 0$$

⑤ Move to S_2

After Step 1	$V(S_1)$	$V(S_2)$	$V(S_3)$	$e(S_1)$	$e(S_2)$	$e(S_3)$
	0	0	0	0.45	0	0

⏱ Step 2: At S_2, move to S_3, reward = 0
① Compute TD Error δ:

$$\delta = 0 + 0.9 \times V(S_3) - V(S_2) = 0 + 0.9 \times 0 - 0 = 0$$

② Boost eligibility of current state S_2:

$$e(S_2) \leftarrow 0 + 1 = 1$$

Traces before update: $e(S_1) = 0.45,\ e(S_2) = 1,\ e(S_3) = 0$
③ Update ALL values:

$$V(S_1) \leftarrow 0 + 0.1 \times 0 \times 0.45 = 0$$

$$V(S_2) \leftarrow 0 + 0.1 \times 0 \times 1 = 0$$

$$V(S_3) \leftarrow 0 + 0.1 \times 0 \times 0 = 0$$

> 💡 $\delta = 0$ again, no updates, but traces are building up!

④ Decay ALL traces:

$$e(S_1) \leftarrow 0.45 \times 0.45 = 0.2025$$

$$e(S_2) \leftarrow 0.45 \times 1 = 0.45$$

$$e(S_3) \leftarrow 0.45 \times 0 = 0$$

⑤ Move to S_3

After Step 2	$V(S_1)$	$V(S_2)$	$V(S_3)$	$e(S_1)$	$e(S_2)$	$e(S_3)$
	0	0	0	0.2025	0.45	0

⑤ Move to S_3

After Step 2	$V(S_1)$	$V(S_2)$	$V(S_3)$	$e(S_1)$	$e(S_2)$	$e(S_3)$
	0	0	0	0.2025	0.45	0

⏱ Step 3: At S_3, move to S_T, reward = +10 ⭐ (THE KEY STEP!)
① Compute TD Error δ:

$$\delta = 10 + 0.9 \times V(S_T) - V(S_3) = 10 + 0.9 \times 0 - 0 = \mathbf{10}$$

② Boost eligibility of current state S_3:

$$e(S_3) \leftarrow 0 + 1 = 1$$

Traces before update: $e(S_1) = 0.2025,\ e(S_2) = 0.45,\ e(S_3) = 1$
③ Update ALL values — this is where the magic happens! ✨

$$V(S_1) \leftarrow 0 + 0.1 \times 10 \times 0.2025 = \mathbf{0.2025}$$

$$V(S_2) \leftarrow 0 + 0.1 \times 10 \times 0.45 = \mathbf{0.45}$$

$$V(S_3) \leftarrow 0 + 0.1 \times 10 \times 1 = \mathbf{1.0}$$

> 🎉 **ALL THREE states updated in a single step!** The reward signal propagated backwards instantly using eligibility traces — unlike TD(0) which would only update S_3 in episode 1.
> `amreis.github`

④ Decay ALL traces:

$$e(S_1) \leftarrow 0.45 \times 0.2025 = 0.0911$$

$$e(S_2) \leftarrow 0.45 \times 0.45 = 0.2025$$

$$e(S_3) \leftarrow 0.45 \times 1 = 0.45$$

⑤ S_T is terminal → Episode ends.

📊 Final Results After Episode 1

State	TD(0) after Episode 1	TD(λ) after Episode 1
$V(S_1)$	0	0.2025 ✅
$V(S_2)$	0	0.45 ✅
$V(S_3)$	1.0	1.0 ✅

The Big Picture: Where We Are

```
RL Control Methods
│
├── Model-Based (know p(s'|s,a))
│     └── Policy Iteration using V(s) + Bellman equations
│
└── Model-Free (don't know the model)
      └── Must use Q(s,a) instead of V(s)
            ├── SARSA   (on-policy TD control)  ← coming soon
            └── Q-Learning (off-policy TD control) ← coming soon
```

What Problem Does Q-Learning Solve?

Imagine you drop a robot in a building it has never seen before. You tell it: "***Find the charging station.***" You give it no map, no instructions — just the ability to move around and receive signals (rewards) when it does something good or bad.

How does the robot figure out the best path?

That's exactly the problem Q-Learning solves. It gives the robot a systematic way to learn from trial and error until it masters the environment.

Part 1: The Q-Table — The Robot's Brain 🧠

The Q-table is simply a spreadsheet. Rows are states (locations), columns are actions (moves). Each cell stores a number — the Q-value — which represents "how good is this move from this location?"

What it starts as

All zeros. The robot knows absolutely nothing.

```text
          Left    Right    Up    Down
Room A  [  0    |   0    |  0  |   0  ]
Room B  [  0    |   0    |  0  |   0  ]
Room C  [  0    |   0    |  0  |   0  ]
Room D  [  0    |   0    |  0  |   0  ]
```

What it becomes after learning

```text
          Left    Right    Up    Down
Room A  [ -1    |  38.6  | -1  |  -1  ]
Room B  [ -1    |   -1   | 50  |  -1  ]
Room C  [ -1    |  44.0  | -1  |  -1  ]
Room D  [ 44    |   -1   | -1  |  -1  ]
```

Part 2: States and Actions — The Building Blocks

State (S)

A state is a complete snapshot of the agent's current situation. In a maze, it's which room you're in. In a chess game, it's the entire board position. In a self-driving car, it's the camera feed, speed, and GPS.

🔑 Rule: The state must contain everything the agent needs to decide what to do next — no hidden information.

Action (A)

An action is a choice the agent makes. In a maze: move left, right, up, down. In a game: which move to play. In a robot: which motor to activate.

The set of all valid actions from a state is called the action space.

Part 3: Rewards — The Feedback Signal 🎁

Rewards are how the environment communicates with the agent. They're the only source of learning.

Types of rewards:

- Positive reward (+) → "Good move! Do this more."
- Negative reward (−) → "Bad move! Avoid this."
- Zero reward → "Neutral. Neither good nor bad."

Reward design matters enormously:

- Reaching the goal: +50 (big positive — we want this!)
- Every step taken: −1 (small negative — encourages efficiency)
- Falling off a cliff: −100 (huge negative — avoid this!)

⚠️ If you design rewards poorly, the agent finds unintended shortcuts. A robot rewarded only for speed might knock over obstacles instead of going around them. This is called reward hacking.

Part 4: The Q-Value — What It Actually Means

Q(s,a) answers this specific question:

"If I am currently in state *s* and I take action *a* right now, what is the total cumulative reward I can expect from this point all the way to the end — assuming I play optimally from here onward?"

This is NOT just the immediate reward. It includes:

- The reward right now
- The reward from the next state
- The reward from the state after that
- ... all the way to the terminal state

🏦 Analogy: Q-value is like the net present value of a business decision. It doesn't just count today's profit — it accounts for all future profits this decision leads to, discounted by how far in the future they are.

Part 5: The Update Formula — Decoded Piece by Piece

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \cdot \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\Big]$$

The TD Error — The "Surprise Signal"

$$\delta_t = R_{t+1} + \gamma \cdot \max_a Q(S_{t+1}, a) - Q(S_t, A_t)$$

This is the most important piece. It measures **how wrong your prediction was**.

- You predicted: $Q(S_t, A_t)$ — what you thought this state-action was worth
- Reality showed: $R_{t+1} + \gamma \cdot \max_a Q(S_{t+1}, a)$ — what actually happened next

The difference between these two = your prediction error. You then **nudge** your Q-value toward the truth by $\alpha \times \delta_t$.

Part 6: ε-Greedy — The Exploration Strategy 🔍

This is the critical piece that lets Q-Learning actually **discover** good paths.

The Exploitation Trap

If the agent always picks the best-known action (greedy), it gets stuck:

- It finds ONE path to the goal early on
- It keeps repeating that same path forever
- It **never discovers** there might be a much better path elsewhere

This is like always eating at the first restaurant you liked — you'll never discover the best restaurant in the city.

The Exploration Trap

If the agent always picks random actions, it never uses what it learned:

- It's constantly thrashing around randomly
- Q-values never stabilize
- It never actually reaches good performance

ε-Greedy — The Perfect Balance

```text
Roll a random number between 0 and 1:
  If number < ε  → Take RANDOM action (Explore 🔍)
  If number ≥ ε  → Take BEST known action (Exploit 💡)
```

ε decay over time (typical schedule):

```text
Early episodes:  ε = 0.9  → 90% random, 10% greedy  (explore a lot)
Mid training:    ε = 0.5  → 50% random, 50% greedy  (balanced)
Late training:   ε = 0.1  → 10% random, 90% greedy  (mostly exploit)
Final:           ε = 0.01 → 1% random, 99% greedy   (nearly optimal)
```

Part 7: Off-Policy — The Superpower of Q-Learning

This is what separates Q-Learning from SARSA and makes it uniquely powerful.

Two Policies in Q-Learning

Policy	Role	Type
Behavior Policy	How the agent actually moves	ε-greedy (sometimes random)
Target Policy	What the agent learns	Always greedy (max)

Why this is powerful

The agent can be **clumsy and exploratory** in its actual behavior, but it always **learns the optimal strategy** through the max operator.

```text
What actually happened:       C → D → B (took random action to B)
What Q-Learning learns from:  "What's the BEST action from D?"
                               Answer: E (because Q(D,E) is highest)
```

Even if the agent went to B by accident, the update for Q(D, ·) uses the value of the BEST next state — not the accidentally-chosen one.

Part 8: Convergence — When Does Q-Learning Stop?

Q-Learning is **guaranteed to converge** to the optimal Q-values $Q^*(s, a)$ if:

1. Every state-action pair is visited **infinitely often** (explore enough!)
2. The learning rate α satisfies: $\sum \alpha = \infty$ and $\sum \alpha^2 < \infty$ (decreases over time but not too fast)

In practice, you just run enough episodes with a gradually decreasing ε, and the Q-table stabilizes.

Signs of convergence:

- Q-values stop changing between episodes
- The agent consistently finds the same (optimal) path
- TD errors become very close to 0

Part 9: From Q-Learning to Deep Q-Learning (DQN)

Classic Q-Learning uses a **table** — it only works when there are a small, finite number of states.

Real-world problems (Atari games, self-driving cars) have **millions or infinite states**. You can't build a table for that.

Solution: Replace the Q-table with a **neural network** that takes state as input and outputs Q-values for all actions.

$$Q(s, a) \approx f_\theta(s, a)$$

This is called **Deep Q-Network (DQN)** — invented by DeepMind in 2013. It's the same Q-Learning update, but instead of updating a table cell, you update the neural network weights using backpropagation.

This is what beat human-level performance on 49 Atari games — the same simple update formula, just scaled up with deep learning.

Numerical

A robot in a factory must navigate from its **start position S1** to a **charging station S4**. There are two possible routes:

- **Route A** (Safe Corridor): S1 → S2 → S4
- **Route B** (Hazard Zone): S1 → S3 → S4

The MDP is defined as follows:

```text
        Safe Corridor
   S1 ──────────────► S2
   │                  │
   │  Hazard Zone     │ R = +10
   │                  ▼
   └─────────────► S3─► S4 (GOAL)
                  R = +2
```

Reward Structure:

Transition	Reward
S1 → S2	−1 (step cost)
S2 → S4	**+10** (goal via safe route)
S1 → S3	−1 (step cost)
S3 → S4	**+2** (goal via hazard zone)

Parameters:

- Learning rate $\alpha = 0.5$
- Discount factor $\gamma = 0.9$
- Initial Q-values: all = 0
- S4 is the **terminal state** → Q(S4, ·) = 0 always

Task: Apply the Q-Learning algorithm for **5 episodes** using the paths given below. Show every TD error, Q-value computation, and table update. At the end, derive the optimal policy.

Episode	Path Taken
1	S1 → S3 → S4 (random exploration)
2	S1 → S2 → S4 (greedy: S1→S2=0 > S1→S3=−0.5)
3	S1 → S3 → S4 (ε-random exploration)
4	S1 → S2 → S4 (ε-random exploration)
5	S1 → S2 → S4 (greedy)

$$Q(S, A) \leftarrow Q(S, A) + \alpha\left[R + \gamma \cdot \max_a Q(S', a) - Q(S, A)\right]$$

🔧 Initial Q-Table (All Zeros)

State	Action →S2	Action →S3	Action →S4
S1	0	0	—
S2	—	—	0
S3	—	—	0

🔁 Episode 1 — Path: S1 → S3 → S4

> Agent explores randomly and takes the hazard zone route.

Step t=1: At S1, take action → S3

- Reward received: R = –1
- Next state: S3
- Available actions from S3: only →S4, so $\max_a Q(S3, a) = Q(S3 \to S4) = 0$

$$\delta = R + \gamma \cdot \max_a Q(S3, a) - Q(S1, S3)$$

$$\delta = -1 + 0.9 \times 0 - 0 = -1$$

$$Q(S1 \to S3) \leftarrow 0 + 0.5 \times (-1) = \boxed{-0.5}$$

Step t=2: At S3, take action → S4 (GOAL)

- Reward received: R = +2
- S4 is terminal → $\max_a Q(S4, a) = 0$

$$\delta = 2 + 0.9 \times 0 - 0 = 2$$

$$Q(S3 \to S4) \leftarrow 0 + 0.5 \times 2 = \boxed{1.0}$$

Q-Table after Episode 1:

State	→S2	→S3	→S4
S1	0	**–0.5**	—
S2	—	—	0
S3	—	—	**1.0**

> 💡 Only states on the visited path updated. S3→S4 learned a reward of +2, and S1→S3 got penalized –0.5.

🔁 Episode 2 — Path: S1 → S2 → S4

> Greedy picks S2 since Q(S1→S2) = 0 > Q(S1→S3) = –0.5.

Step t=1: At S1, take action → S2

- R = –1
- $\max_a Q(S2, a) = Q(S2 \to S4) = 0$

$$\delta = -1 + 0.9 \times 0 - 0 = -1$$

$$Q(S1 \to S2) \leftarrow 0 + 0.5 \times (-1) = \boxed{-0.5}$$

Step t=2: At S2, take action → S4 (GOAL)

- R = +10
- $\max_a Q(S4, a) = 0$

$$\delta = 10 + 0.9 \times 0 - 0 = 10$$

$$Q(S2 \to S4) \leftarrow 0 + 0.5 \times 10 = \boxed{5.0}$$

Q-Table after Episode 2:

State	→S2	→S3	→S4
S1	**–0.5**	–0.5	—
S2	—	—	**5.0**
S3	—	—	1.0

> 💡 Both paths from S1 now have Q = –0.5. The big reward of +10 has been stored in Q(S2→S4)=5, but hasn't yet propagated back to S1. The next step is crucial!

🔁 Episode 3 — Path: S1 → S3 → S4

> ε-random exploration forces S3 path again (despite tie at –0.5).

Step t=1: At S1, take action → S3

- R = –1
- $\max_a Q(S3, a) = Q(S3 \to S4) = 1.0 \leftarrow$ now nonzero from Episode 1!

$$\delta = -1 + 0.9 \times 1.0 - (-0.5)$$

$$\delta = -1 + 0.9 + 0.5 = 0.4$$

$$Q(S1 \to S3) \leftarrow -0.5 + 0.5 \times 0.4 = -0.5 + 0.2 = \boxed{-0.3}$$

Step t=2: At S3, take action → S4

- R = +2

$$\delta = 2 + 0.9 \times 0 - 1.0 = 1.0$$

$$Q(S3 \to S4) \leftarrow 1.0 + 0.5 \times 1.0 = \boxed{1.5}$$

Q-Table after Episode 3:

State	→S2	→S3	→S4
S1	–0.5	**–0.3**	—
S2	—	—	5.0
S3	—	—	**1.5**

> ⚠️ Notice: Q(S1→S3) = –0.3 is now *greater* than Q(S1→S2) = –0.5. Without exploration, the agent would keep choosing the hazard zone — even though S2 holds a reward of 5.0 waiting to be used! This is exactly why ε-greedy exploration matters.

🔁 Episode 4 — Path: S1 → S2 → S4

> ε-random forces agent to try S2 again — and now Q(S2→S4) = 5.0 is used! ⭐

Step t=1: At S1, take action → S2

- R = −1
- $\max_a Q(S2, a) = Q(S2 \to S4) = 5.0 \leftarrow$ THE KEY UPDATE!

$$\delta = -1 + 0.9 \times 5.0 - (-0.5)$$

$$\delta = -1 + 4.5 + 0.5 = 4.0$$

$$Q(S1 \to S2) \leftarrow -0.5 + 0.5 \times 4.0 = -0.5 + 2.0 = \boxed{1.5}$$

Step t=2: At S2, take action → S4

- R = +10

$$\delta = 10 + 0 - 5.0 = 5.0$$

$$Q(S2 \to S4) \leftarrow 5.0 + 0.5 \times 5.0 = 5.0 + 2.5 = \boxed{7.5}$$

Q-Table after Episode 4:

State	→S2	→S3	→S4
S1	**1.5**	−0.3	—
S2	—	—	**7.5**
S3	—	—	1.5

> 🎯 **Big insight!** Q(S1→S2) jumped from −0.5 to +1.5 in a single episode! Now greedy will always pick S2 (1.5 >> −0.3). The agent has discovered the safe corridor is far superior.

🔁 Episode 5 — Path: S1 → S2 → S4 (Greedy)

> Agent now consistently chooses the safe corridor. Watching convergence.

Step t=1: At S1, take action → S2 (greedy: 1.5 >> −0.3)

- R = −1
- $\max_a Q(S2, a) = 7.5$

$$\delta = -1 + 0.9 \times 7.5 - 1.5$$

$$\delta = -1 + 6.75 - 1.5 = 4.25$$

$$Q(S1 \to S2) \leftarrow 1.5 + 0.5 \times 4.25 = 1.5 + 2.125 = \boxed{3.625}$$

Step t=2: At S2, take action → S4

- R = +10

$$\delta = 10 + 0 - 7.5 = 2.5$$

$$Q(S2 \to S4) \leftarrow 7.5 + 0.5 \times 2.5 = 7.5 + 1.25 = \boxed{8.75}$$

Q-Table after Episode 5:

State	→S2	→S3	→S4
S1	**3.625**	−0.3	—
S2	—	—	**8.75**
S3	—	—	1.5

📊 Complete Episode-wise Convergence Tracker

Episode	Q(S1→S2)	Q(S1→S3)	Q(S2→S4)	Q(S3→S4)
Start	0	0	0	0
After Ep 1	0	−0.5	0	1.0
After Ep 2	−0.5	−0.5	5.0	1.0
After Ep 3	−0.5	−0.3	5.0	1.5
After Ep 4	**1.5**	−0.3	7.5	1.5
After Ep 5	**3.625**	−0.3	**8.75**	1.5
True Optimal	**8.0**	**0.8**	**10.0**	**2.0**

🏆 Optimal Policy Extraction

Pick the action with the highest Q-value at each state:

State	Best Action	Q-value	Alternative	Q-value
S1	→ **S2** (Safe Corridor)	**3.625** → converging to 8.0	→ S3	−0.3
S2	→ **S4** (Goal)	**8.75** → converging to 10.0	—	—

Optimal Path: S1 → S2 → S4 ✅

🔑 Key Observations

- **Episode 1–2**: Agent discovered both paths exist. Rewards stored locally in Q-table but not yet propagated to S1.
- **Episode 3**: Hazard zone briefly looked attractive (Q = −0.3 > −0.5), showing why **ε-greedy exploration is critical** — without it, the agent might get stuck.
- **Episode 4**: Forced exploration of S2 caused a massive update (−0.5 → +1.5) by using the already-stored Q(S2→S4) = 5.0. This is the **reward propagating backwards**.
- **Episode 5+**: Agent firmly commits to safe corridor. Q-values converge toward theoretical optimum (Q*=8, Q*=10).
- The **hazard zone path** is permanently abandoned — Q(S1→S3) = −0.3 far below Q(S1→S2) = 3.625 and growing.

SARSA - Part 1: The Name Tells You Everything

State → Action → Reward → State → Action

This is literally the sequence of 5 things that happen at every single step. Let's trace through each one:

```text
You are in STATE S
      ↓
You take ACTION A (using ε-greedy)
      ↓
Environment gives you REWARD R
      ↓
You land in new STATE S'
      ↓
You choose your NEXT ACTION A' (using ε-greedy again)
      ↓
NOW you update Q(S, A) using all 5 things
```

The crucial insight: **you must choose the next action A' BEFORE updating**. This is what makes SARSA on-policy.

Part 2: What Problem Does SARSA Solve?

Imagine a robot learning to navigate a warehouse. It needs to:

- Pick up packages (get rewards)
- Avoid bumping into workers (big negative reward)
- Find efficient routes

The robot doesn't know the layout. It must learn purely by doing. SARSA is the algorithm that lets it:

- Learn from **every single step** (not just end of episode)
- Account for the fact that it **sometimes takes random exploratory steps**
- Be **conservative near dangerous areas** because it knows it might slip

Part 3: The Q-Table in SARSA

Just like Q-Learning, SARSA maintains a Q-table where each cell $Q(s, a)$ answers:

> *"If I am in state* s*, take action* a*, and then follow my current* ε*-greedy policy from that point forward — what total reward can I expect?"*

The key difference from Q-Learning is that last part: **"follow my current ε-greedy policy"** — not the best possible policy.

This means SARSA's Q-values represent the value of being a **real, imperfect, sometimes-random agent** — not a hypothetical perfect one.

Part 4: The Update Formula — Every Piece Decoded

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \cdot Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\Big]$$

The TD Error in SARSA

$$\delta_t = R_{t+1} + \gamma \cdot Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$$

The TD error is still the "surprise signal" — how wrong your prediction was. But what you compare against is different:

- **Q-Learning TD error:** $R + \gamma \cdot \max_a Q(S', a) - Q(S, A)$ — uses the BEST future
- **SARSA TD error:** $R + \gamma \cdot Q(S', A') - Q(S, A)$ — uses the ACTUAL future

Walking Through the Formula Piece by Piece

$Q(S_t, A_t)$ — Your current rating for "being at state S and taking action A"

R_{t+1} — The actual reward you just received. This is real, ground-truth data from the environment.

$\gamma \cdot Q(S_{t+1}, A_{t+1})$ — The discounted value of what happens next. But note: A_{t+1} was chosen by **ε-greedy** — so sometimes it's the best action, sometimes it's a random one. This is the honest accounting of your real future behavior.

$[\ldots] - Q(S_t, A_t)$ — The gap between what actually happened and what you predicted. If positive, you were pleasantly surprised → increase Q. If negative, things went worse than expected → decrease Q.

$\alpha \times \delta_t$ — Don't fully trust one experience. Take a small step in the direction of the correction.

Part 5: On-Policy — The Deep Meaning

"On-policy" means the **learning policy** and the **behavior policy** are the same.

Think of it as two roles an agent can have:

Role	Question it answers
Behavior Policy	"What action do I actually take right now?"
Target Policy	"What action do I learn from?"

In SARSA, both roles are filled by the **same ε-greedy policy**:

```text
At state S:
  ε-greedy picks action A  ← behavior policy (actually taken)

At state S':
  ε-greedy picks action A' ← target policy (used in update)

Same policy! → On-Policy
```

In Q-Learning:

```text
At state S:
  ε-greedy picks action A  ← behavior policy (actually taken)

In the update:
  max_a is used            ← target policy (greedy, different from behavior)

Different policies! → Off-Policy
```

Part 6: The Cautious Nature of SARSA

Consider this situation: The agent is at a state near a cliff. Actions available: Step Left (safer), Step Right (toward cliff).

Q-values: Q(S, Left) = 3, Q(S, Right) = 5

Q-Learning's perspective:

> "I'll learn Q(S, Right) = 5 as the target. In training, my ε-greedy behavior sometimes randomly steps off the cliff. But that's fine — the update still assumes I'll act perfectly (max) in the future."

Q-Learning happily learns the optimal but risky path. During training, it keeps falling off the cliff due to random exploration. It learns the right answer theoretically, but suffers a lot during training.

SARSA's perspective:

> "I know my ε-greedy policy will sometimes randomly step off the cliff. So I'll bake that risk into my Q-values. Q(S, Right) will be lower because I know I occasionally make random moves near the cliff."

SARSA learns to avoid the cliff edge entirely — taking a longer but safer path — because it honestly accounts for its own exploratory behavior.

> ⛰️ **Real-world impact**: In robotics, a robot using Q-Learning might walk close to table edges (optimal path) but keep falling off during training. A robot using SARSA learns to stay away from edges because it knows it sometimes drifts.

```
──────────────────────────────────────────
SARSA ALGORITHM
──────────────────────────────────────────
SETUP:
  Initialize Q(s,a) = 0 for all s, a
  Set Q(terminal, ·) = 0

FOR each episode:
  ┌─ Initialize starting state S
  │
  │  Choose A from S using ε-greedy  ← IMPORTANT: pick A before the loop!
  │
  │  REPEAT for each step:
  │  ┌─ Take action A
  │  │  Observe reward R and next state S'
  │  │
  │  │  Choose A' from S' using ε-greedy  ← pick NEXT action NOW
  │  │
  │  │  Update Q(S,A) ← Q(S,A) + α[R + γQ(S',A') − Q(S,A)]
  │  │
  │  │  S ← S'  (move to next state)
  │  │  A ← A'  (carry forward the chosen action)
  │  └─
  └─ UNTIL S is terminal
──────────────────────────────────────────
```

Step	Q-Learning	SARSA
Choose A	ε-greedy	ε-greedy
Take A, observe R, S'	Same	Same
Update formula	Uses $\max_a Q(S', a)$	Uses $Q(S', A')$ where A' chosen by ε-greedy
Next step	Move to S'	Move to S', **carry A' forward**

Part 7: Variance and Stability

SARSA is generally **more stable** during training because:

- It never over-estimates Q-values by assuming perfect future behavior
- Updates are smoother — they reflect the actual expected return under the current policy
- Near dangerous states, it naturally becomes conservative

Q-Learning can **overestimate** Q-values because it always uses the max — and the max of noisy estimates is biased upward (this is a known problem called "maximization bias," which led to the invention of Double Q-Learning).

Part 8: When to Use SARSA vs Q-Learning

Use SARSA when...	Use Q-Learning when...
Safety during training matters (robotics, medical AI)	You only care about final performance
The environment has dangerous states with large penalties	You train in a simulation (falling doesn't matter)
The task is stochastic (random transitions)	You want to converge to true optimal faster
You want stable, smooth training curves	Training efficiency is more important than safety
Real hardware is involved (can't afford failures)	Playing games, virtual environments

SARSA(λ) — The Powerful Extension

Just like TD(0) was extended to TD(λ), SARSA can be extended to **SARSA(λ)** using eligibility traces:

$$Q(S, A) \leftarrow Q(S, A) + \alpha \cdot \delta_t \cdot e_t(S, A)$$

Where $e_t(S, A)$ is the eligibility trace for state-action pair (S, A).

This means SARSA(λ) propagates the TD error **backwards to all recently visited state-action pairs** — exactly like TD(λ) did for state values. It learns faster because credit assignment happens across multiple steps simultaneously.

Numerical